

Core_{sk}

ANONYMOUS AUTHOR(S)

In recent years, significant advances have been made in numerical debugging and static analysis tools. Recent debugging tools significantly reduce overhead using double-double oracles, while recent static analysis tools accurately bound rounding errors using condition numbers. However, these new techniques also have downsides: double-double oracles can miss floating-point errors due to correlated errors, while condition number formalisms cannot cleanly handle over- and underflow. We show that combining both techniques avoids the downsides of each and implement this combination in EASTEREGG. EASTEREGG uses double-double computations but not as an oracle; instead, it detects erroneous operations using condition numbers, which minimizes overhead while achieving precision and recall similar to more expensive methods. EASTEREGG also introduces a separate oracle to detect harmful over- and underflows more accurately; we implement this oracle in a new performant number representation. As a result, EASTEREGG achieves a precision of 80.0% and a recall of 96.1% on a collection of 500 difficult numeric benchmarks. An ablation study shows that EASTEREGG combines the strengths of both double-double oracles and condition numbers: it is more accurate than double-double oracles yet dramatically faster than methods based on arbitrary-precision condition number computations.

ACM Reference Format:

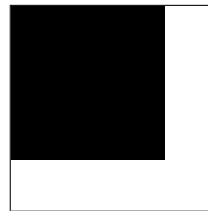
Anonymous Author(s). 2026. Core_{sk}. 1, 1 (February 2026), 5 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

2 Skia

In this section, we introduce the basics of the Skia's drawing API. The next example demonstrates the API by drawing a rectangle. All drawing happens into a canvas.¹

Draw(Rect1, DEFAULT_PAINT)



The example is shown in three parts: the canvas before the draw call, the draw command itself, and the canvas after execution. The drawing before the Draw command is an empty pixel buffer. The drawing after the Draw command is a pixel buffer with a black square. Draw needs 2 arguments: the geometry to be drawn, and a Paint.²

¹A canvas in Skia is represented by an instance of a SkCanvas [1] object, which owns all the drawing state.

²In the actual SkCanvas API, Draw is not overloaded over all geometries, but instead there exist specific Draw* methods for the geometries Skia supports.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

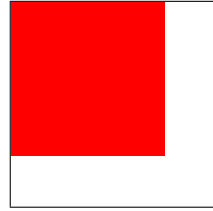
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/2-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

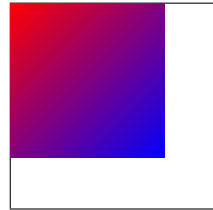
A `Paint` [2] controls how a geometry is drawn. For example, you can specify a fill color in a `Paint`.

```
Draw(Rect1,
    Paint(Color(Red), ...))
```



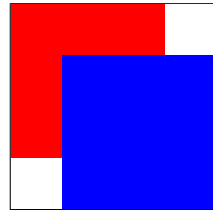
A fill does not need to be a constant fill of the same pixel. More interesting fills can be implemented with gradients:

```
Draw(Rect1,
    Paint(LinearGradient(...), ...))
```



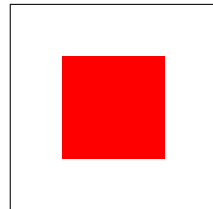
You can also specify a blend mode for a `Paint`, which governs how the geometry is drawn on to existing pixels of the canvas. For example, the blend mode `SrcOver` encodes the intuitive idea of drawing one shape on top of the other. In the following example, we blend a blue square over the red square we previously drew, using `SrcOver` blending.

```
Draw(Rect2,
    Paint(Color(Blue), ..., SrcOver))
```



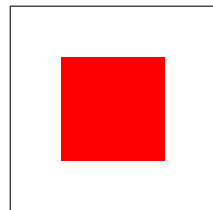
Another blend mode common blend mode is `DstIn`. `DstIn`. If a geometry is blended with `DstIn`, instead of being drawn on top the canvas, like in `SrcOver`, it is instead used as mask over the canvas. Consider the previous example, drawn using `DstIn` instead of `SrcOver`.

```
Draw(Rect2,
    Paint(Color(Blue), ..., DstIn))
```



While `DstIn` can be used to mask drawings in this way, Skia also provides a dedicated clipping command for expressing this operation more directly.

```
Clip(Rect2, Intersect);
Draw(Rect1,
    Paint(Color(Red), ..., SrcOver))
```



```

Image  i   : Point → Color
Shape  g   : Point → bool
Blend  b   : Color × Color → Color

```

Fig. 1. Our abstract model consists of Images and Shapes over abstract types Pixels and Colors, which can be blended together to form images.

```

CoreSkia  s ::= l
Layer     l ::= Empty()
           | Draw(l, g: Geometry, p: Paint)
           | SaveLayer(l, l, p: Paint)

```

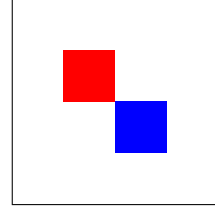
Fig. 2

Now instead masking being a consequence of blending, we mask by constraining the draw geometry by intersecting it with a clip. The current global clip state is maintained by the canvas, and Clip commands modify the global clip state.

```

Clip(Rect(64, 64, 192, 192), Intersect);
Draw(Rect(0, 0, 128, 128),
    Paint(Color((1, 1, 0, 0)), ..., SrcOver));
Draw(Rect(128, 128, 256, 256),
    Paint(Color((1, 0, 0, 1)), ..., SrcOver))

```

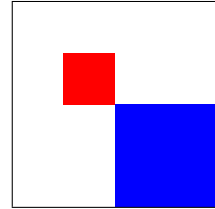


Skia does not maintain just one clip state, but rather a stack of them. The current canvas clip state is the clip on the top of the clip stack. Clip commands always modify the top of the clip stack, and can not touch the states below it. Skia provides the Save() and Restore() commands to manipulate the clip state. Save() duplicates the top of the clip stack on to the stack, and a matching Restore() pops the top of the stack. Save() and Restore() allow us to restrict the effects of Clip commands to a group of commands.

```

Save();
Clip(Rect(64, 64, 192, 192), Intersect);
Draw(Rect(0, 0, 128, 128),
    Paint(Color((1, 1, 0, 0)), ..., SrcOver));
Restore();
Draw(Rect(128, 128, 256, 256),
    Paint(Color((1, 0, 0, 1)), ..., SrcOver))

```



3 Semantics

Our semantics of Skia is built around a simple abstract model of images and shapes, shown in fig:model. - Abstract type of Pixels and Colors - Image is Pixel → Color - Shape is Pixel → Bool - Blend is Color → Color → Color Skia works by taking Shapes; filling them with primitive Images; Blending the results together in various orders

However, Skia does not permit arbitrary drawing and blending operations; instead, these always happen in a prescribed order. We thus introduce the Layer abstraction, shown in fig:layer, which defines the core Draw and BlendLayer operations in Skia. We focus on these two operations specifically because they are the core target of the optimizing compiler described in sec:optimize.

The Layer constructors can be defined in terms of the abstract model, as shown in fig:denote. -
 Rename SaveLayer to BlendLayers(l, l, p) - Define our 3 constructs in terms of primitive operations

We can now move on to defining the grammar of Skia.

As our main goal is to try and verify rewrites in Skia, it is hard to do so over Surface_{Skia}. This is mainly because of the hidden layer and clip state of Surface_{Skia}. That is why we introduce a new language called Core_{Skia}, which is a declarative version of Surface_{Skia}. Core_{Skia} makes layers the main value type of the language, and also make explicit the clip state at each draw command. This also has the consequence of completely removing Saves from Core_{Skia}.

We need to talk about how to convert Surface_{Skia} to Core_{Skia}. Not sure how to do that.

To verify rewrites in Core_{Skia} we define semantics for it. We define a denotation function $\llbracket \cdot \rrbracket$ over Layers.

$$\llbracket \text{Layer} \rrbracket : \text{Point} \rightarrow \text{Pixel}$$

That is, A Layer is a map from Points to Pixels, or more literally a buffer of pixels, indexed by points. We now define the denotation function over the 3 types of Layers. First, the Empty layer.

$$\llbracket \text{Empty}() \rrbracket = \lambda pt.(0, 0, 0, 0)$$

Next, Draw.

$$\llbracket \text{Draw}(l, g, p, c) \rrbracket =$$

```
let l_b := denote l_b
let (fill, blend_mode, style, filter) := paint
let drawPx := if (style geo) pt && (clip pt)
               then ((getFillFunc fill) pt)
               else Pixel.Transparent
blend_mode (l_b pt) (filter drawPx)
```

idk how to typeset this man

Next, Draw.

$$\llbracket \text{SaveLayer}(l_b, l_t, g, p) \rrbracket =$$

```
let l_b := denote l_b
let l_t := denote l_t
let blend_mode : BlendMode := paint.2.1
let filter      : Filter    := paint.2.2.2
let alpha       : Real      := getAlpha paint.1
blend_mode (l_b pt) (filter (applyAlpha alpha (l_t pt)))
```

Argument symbols are all over the place. There is applyAlpha here for som god forsaken reason. I am very sleepy.

4 Compiler

5 Evaluation

6 Related Work

7 Conclusion

References

[1] [n. d.]. Skia: SkCanvas Class Reference. <https://api.skia.org/classSkCanvas.html>. Accessed: 2026-02-06.

[2] [n. d.]. Skia: SkPaint Class Reference. <https://api.skia.org/classSkPaint.html>. Accessed: 2026-02-06.